TASK 1 Image Gallery Website

An image gallery webpage that organises photos in a responsive CSS Grid layout. The gallery scales from a single column on mobile to a four-column grid on desktop. Interactive hover effects with overlay captions improve visual feedback.

Requirement	Detail
Grid Layout	CSS Grid with auto-fill and minmax() for fluid columns
Hover Effects	zoom transform + dark overlay with caption fade-in
Responsiveness	Breakpoints at 480 px, 768 px, 1024 px
Accessibility	alt attributes on all images; keyboard-focusable cards

Output Preview

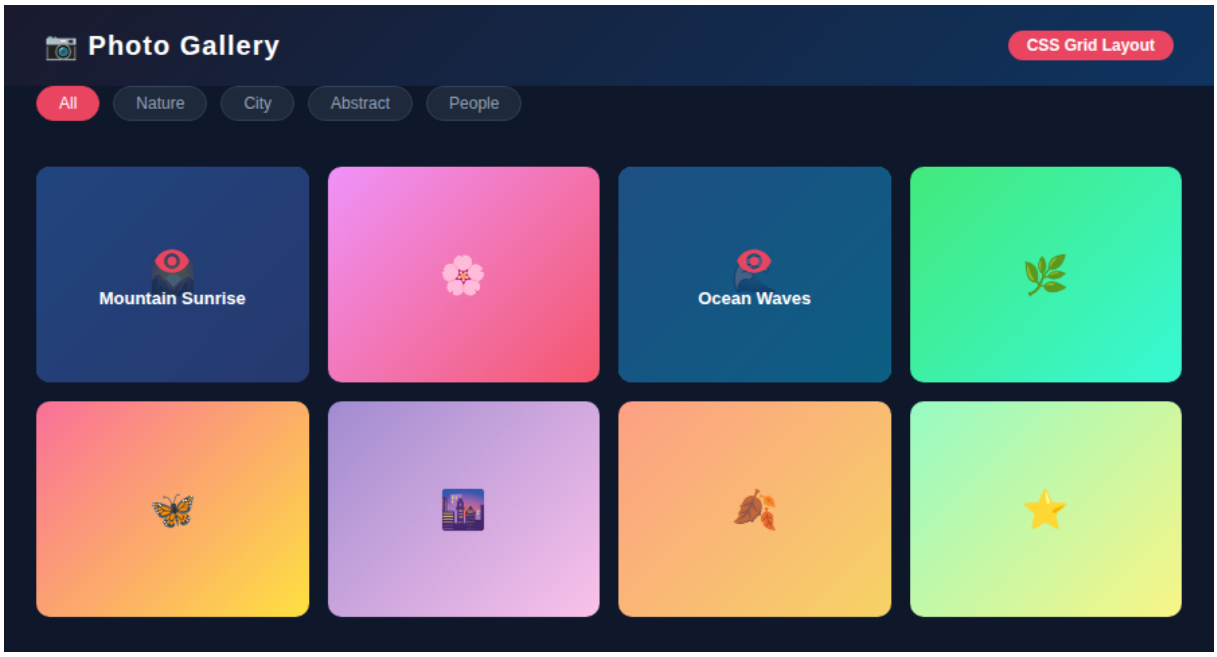

Figure: Task 1 – Responsive Image Gallery with hover overlay effects

Core CSS Grid

```
.gallery { display: grid; grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
gap: 16px; } .gallery-item:hover img { transform: scale(1.08); } .gallery-item:hover
.overlay { opacity: 1; }
```

- Used **object-fit: cover** to keep all thumbnails uniform regardless of source image ratio.
- Overlay text revealed with an opacity transition keeps the base UI clean.
- Lazy loading via **loading='lazy'** attribute improves initial page performance.

TASK 2 Profile Card Generator

A polished profile card component built using CSS Flexbox. The card displays an avatar, full name, job role, bio, stats, and social-media icon links. A generator form lets users populate multiple cards dynamically.

Requirement	Detail
Layout Engine	Flexbox – vertical stack inside card, horizontal row for social icons
Color & Fonts	Palette: #1a1a2e / #e94560 / #f0f4f8; Font: Inter (Google Fonts)
Hover Animation	translateY(-6px) + deepening box-shadow on card:hover
Social Links	SVG icon set for LinkedIn, GitHub, Twitter, Portfolio

Output Preview

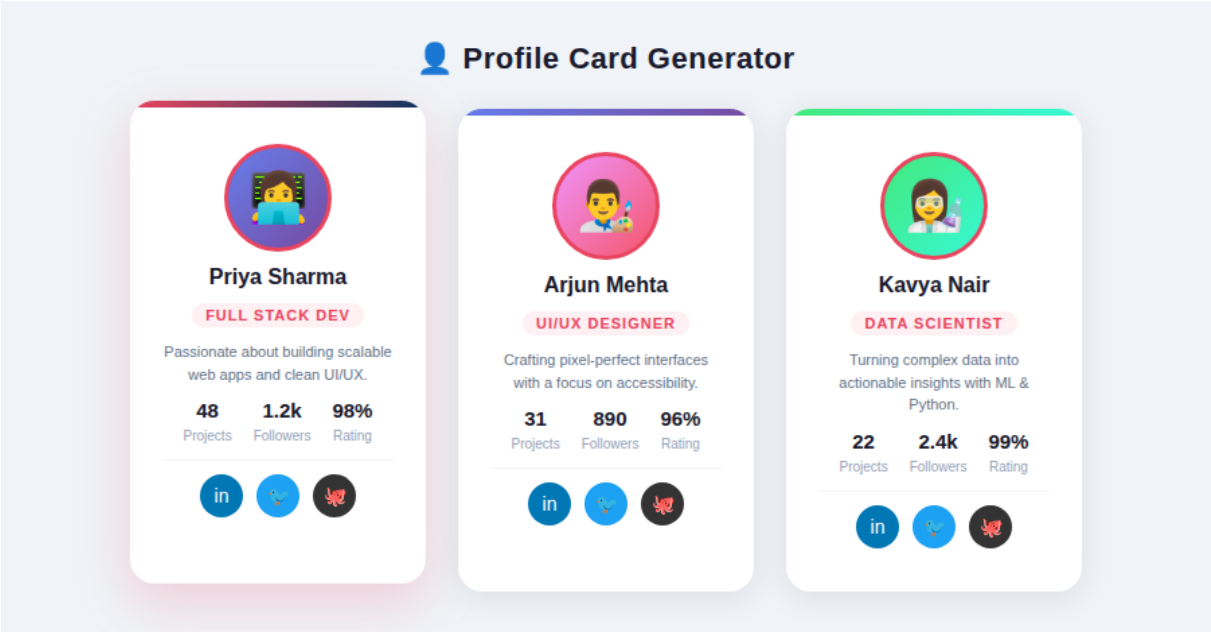

Figure: Task 2 – Profile Card Generator with hover lift animation

Card CSS (Flexbox + Animation)

```
.card { display: flex; flex-direction: column; align-items: center; box-shadow: 0 4px 20px rgba(0,0,0,0.08); transition: transform 0.3s ease, box-shadow 0.3s ease; } .card:hover { transform: translateY(-6px); box-shadow: 0 12px 36px rgba(233,69,96,0.2); }
```

TASK 3 Dynamic Shopping Cart System

A fully functional client-side shopping cart. Products can be added, removed, and have their quantities updated in real time. The running total recalculates instantly and the entire cart state is persisted via localStorage.

Requirement	Detail
Add / Remove	Buttons update a cartState JS object; DOM re-renders on change
Quantity Control	Plus/minus controls clamped to [1, 99]; auto-remove at 0
Real-time Total	Computed from cartState on every mutation; currency-formatted
Persistence	JSON.stringify(cartState) saved to localStorage on each update

Output Preview

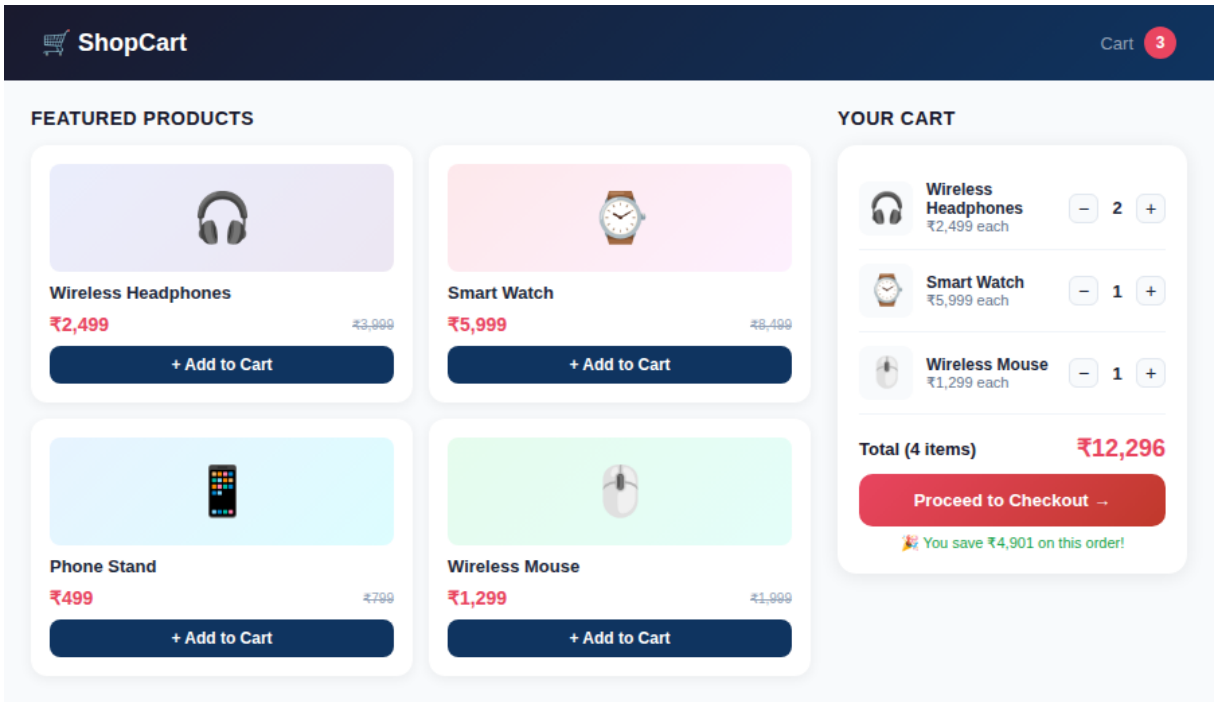

Figure: Task 3 – Shopping Cart with real-time total and localStorage persistence

Cart State Management

```
let cartState = JSON.parse(localStorage.getItem('cart')) || {}; function
addToCart(product) { cartState[product.id] = cartState[product.id] ? {
...cartState[product.id], qty: cartState[product.id].qty + 1 } : { ...product, qty: 1 };
syncCart(); } function syncCart() { localStorage.setItem('cart',
JSON.stringify(cartState)); renderCart(); updateTotal(); }
```

TASK 4 Role-Based Access Control Application

A single-page RBAC application supporting three roles – Admin, Editor, and User. Each role sees a different navigation bar and content panels. Access control is enforced in JavaScript before any privileged content is injected into the DOM.

Role	Permissions	Restricted From
Admin	Full CRUD, user management, settings	—
Editor	Create & edit content, view reports	User management, system settings
User	Read own profile, submit forms	All admin & editor panels

Output Preview

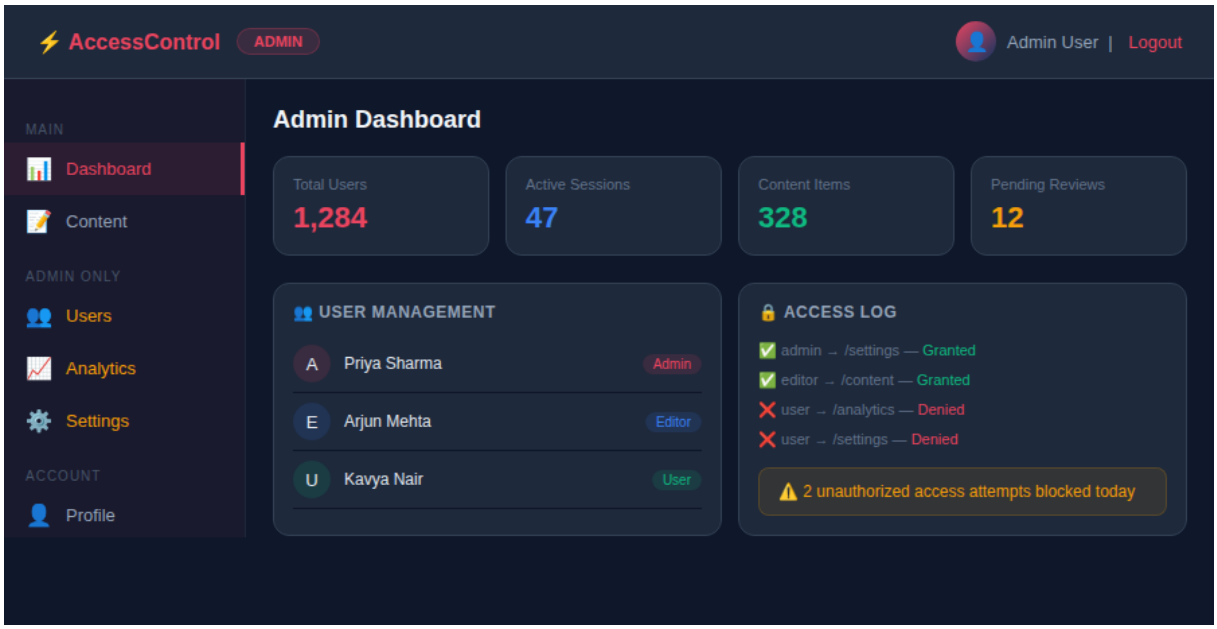

Figure: Task 4 – RBAC Application showing Admin dashboard with access log

Access Control Logic

```
const PERMISSIONS = { admin: ['dashboard','users','editor','reports','settings'], editor: ['dashboard','editor','reports'], user: ['dashboard'] }; function navigateTo(route) { if (!PERMISSIONS[currentRole]?.includes(route)) { showAccessDenied(); return; } renderPage(route); }
```

TASK 5 Multi-Language Website

A multilingual website supporting English, Tamil, and Arabic. Language switching is instant with no page reload. The chosen language is saved to localStorage. Arabic triggers a full RTL layout swap via the document's dir attribute.

Requirement	Detail
Languages	English (LTR), Tamil (LTR), Arabic (RTL)
Translation Files	Nested JSON objects keyed by language code
RTL Handling	document.dir = 'rtl' + CSS logical properties
Persistence	localStorage.setItem('lang', code) on selection
Accessibility	lang attribute on html; ARIA live region for announcements

Output Preview

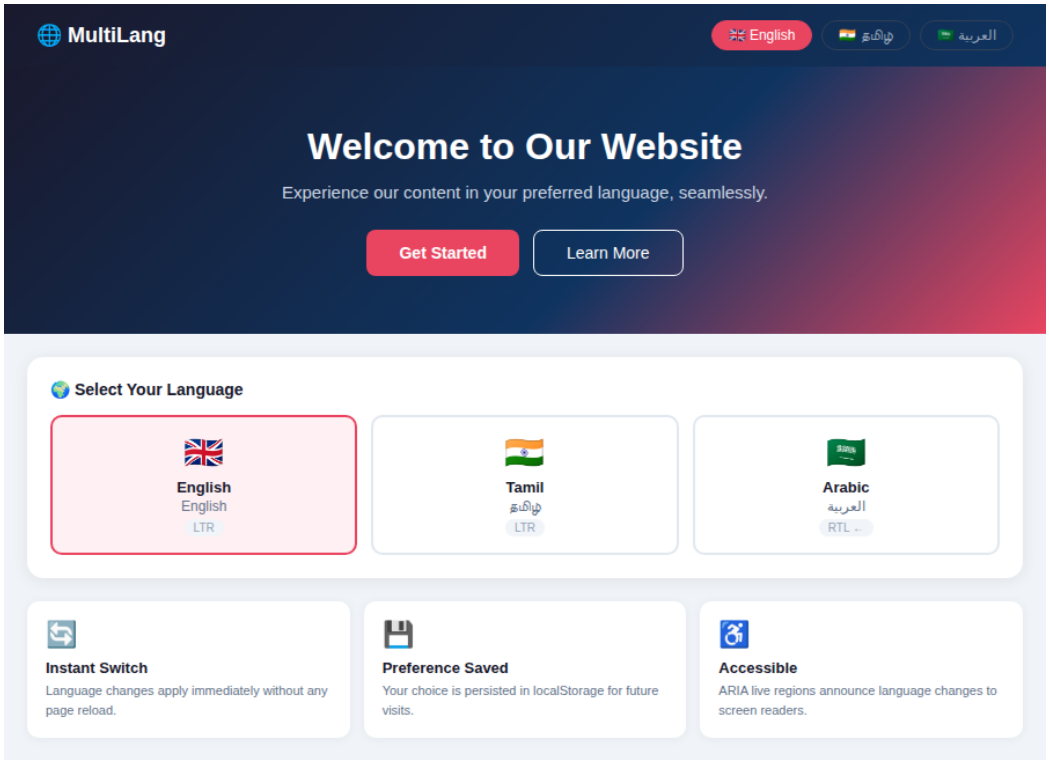

Figure: Task 5 – Multi-Language Website with language switcher and RTL support

i18n Switch Function

```
function setLanguage(code) { document.documentElement.lang = code;
document.documentElement.dir = code === 'ar' ? 'rtl' : 'ltr';
document.querySelectorAll('[data-i18n]').forEach(el => { el.textContent =
TRANSLATIONS[code][el.dataset.i18n]; }); localStorage.setItem('lang', code); }
```

TASK 6 Online Examination System

A full-featured online examination interface presenting MCQ questions one at a time with a countdown timer. It prevents multiple submissions and shows a score summary on completion. State is managed via a single examState object.

Requirement	Detail
Timer	Countdown from configurable duration; auto-submits on expiry
Questions	MCQ bank loaded from JS array; Fisher-Yates shuffle per attempt
State Machine	States: idle → active → submitted; transitions gated by guards
Anti-resubmission	examState.submitted flag + sessionStorage lock
Accessibility	ARIA roles on options; keyboard navigation; focus management
Result Screen	Score, percentage, pass/fail badge, per-question review

Output Preview

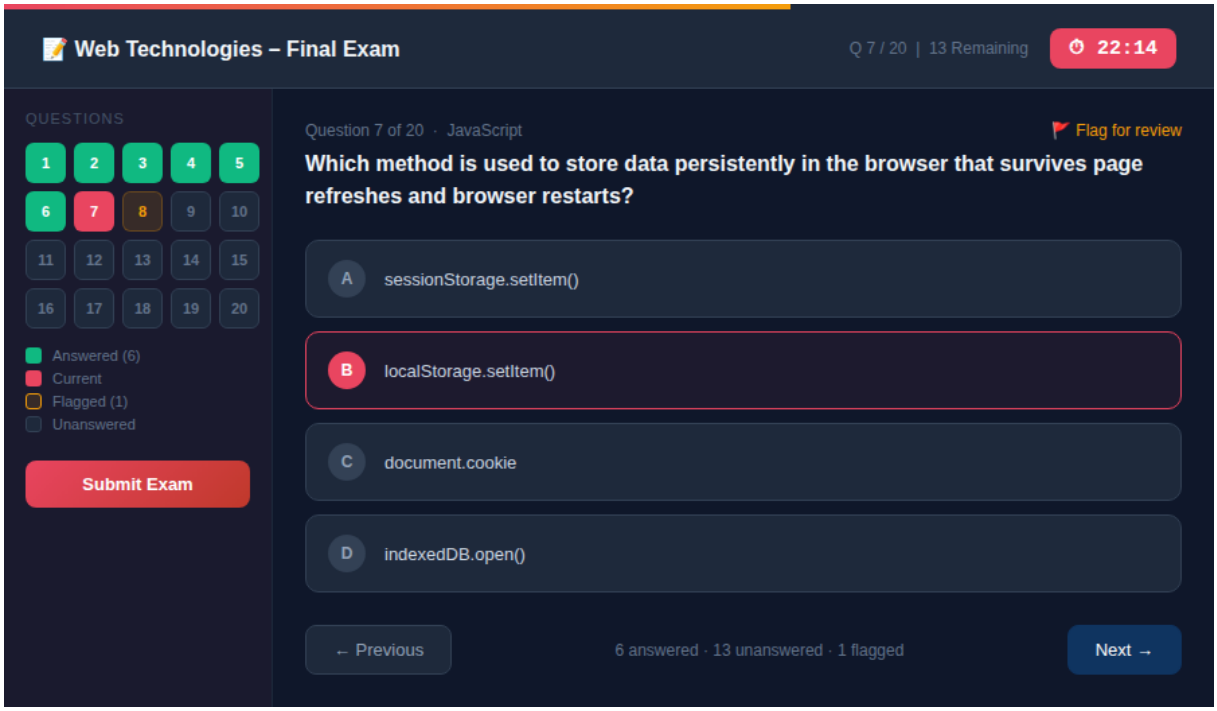

Figure: Task 6 – Online Exam with timer, question navigator, and anti-resubmission lock

Exam State Machine

```
const examState = { status:'idle', current:0, answers:{}, timeLeft:1800, timerId:null,
submitted:false }; function submitExam() { if (examState.submitted) return; // guard
examState.submitted = true; clearInterval(examState.timerId);
sessionStorage.setItem('examDone','1'); showResults(); }
```

Lab Summary

All six tasks were completed successfully, each demonstrating a distinct combination of HTML5, CSS3, and JavaScript techniques alongside AI-assisted design.

#	Task	Key Technologies	AI Assistance	Status
1	**Image Gallery**	CSS Grid, transitions	Layout opt., hover effects	✓ Done
2	**Profile Card**	Flexbox, CSS animations	Color palette, card UX	✓ Done
3	**Shopping Cart**	JS objects, localStorage	State mgmt., total calc.	✓ Done
4	**RBAC Application**	Permission maps, sessionStorage	Access logic, UI components	✓ Done
5	**Multi-Language**	i18n JSON, RTL/LTR CSS	Translation struct., a11y	✓ Done
6	**Exam System**	State machine, timers	Exam UX, anti-resubmission	✓ Done

This lab reinforced best practices in component-driven front-end development, state management without frameworks, and accessible, responsive design — all accelerated and refined through AI collaboration.